

7. APPENDIX

7.1 PACMAN.HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Pacman</title>
```

```
</head>
```

```
<body style="margin: 0; background-color: black;">
```

```
<canvas id="canvas" width="500" height="500"></canvas>
```

```
<div style="display:none;">
```

```

```

```

```

```
</div>
```

```
<script src="ghost.js"></script>
```

```
<script src="pacman.js"></script>
```

```
<script src="game.js"></script>
```

```
</body>
```

```
</html>
```

7.2 GAME.JS

```
const canvas = document.getElementById("canvas");
```

```
const canvasContext = canvas.getContext("2d");
```

```
const pacmanFrames = document.getElementById("animation");
```

```
const ghostFrames = document.getElementById("ghosts");
```

```
let createRect = (x, y, width, height, color) => {  
  canvasContext.fillStyle = color;  
  canvasContext.fillRect(x, y, width, height);  
};
```

```
const DIRECTION_RIGHT = 4;
```

```
const DIRECTION_UP = 3;
```

```
const DIRECTION_LEFT = 2;
```

```
const DIRECTION_BOTTOM = 1;
```

```
let lives = 3;
```

```
let ghostCount = 4;
```

```
let ghostImageLocations = [  
  { x: 0, y: 0 },
```

```
  { x: 176, y: 0 },
```

```

    { x: 0, y: 121 },
    { x: 176, y: 121 },
];

// Game variables

let fps = 30;

let pacman;

let oneBlockSize = 20;

let score = 0;

let ghosts = [];

let wallSpaceWidth = oneBlockSize / 1.6;

let wallOffset = (oneBlockSize - wallSpaceWidth) / 2;

let wallInnerColor = "black";


// we now create the map of the walls,

// if 1 wall, if 0 not wall

// 21 columns // 23 rows

let map = [

    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [1, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 1],
    [1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1, 1, 2, 1],
    [1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1, 1, 2, 1],
    [1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1],
    [1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1],
    [1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 1],

```

```

[1, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1, 1, 1, 1, 1],
[0, 0, 0, 0, 1, 2, 1, 2, 2, 2, 2, 2, 2, 1, 2, 1, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 2, 1, 2, 1, 1, 2, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1],
[1, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 1],
[1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 2, 2, 1, 2, 1, 2, 1, 1, 1, 1, 1],
[0, 0, 0, 0, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 1, 0, 0, 0, 0],
[0, 0, 0, 0, 1, 2, 1, 2, 2, 2, 2, 2, 2, 1, 2, 1, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1],
[1, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1],
[1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1],
[1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1],
[1, 1, 2, 2, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 2, 1, 1],
[1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1],
[1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1],
[1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
];

```

```

let randomTargetsForGhosts = [
  { x: 1 * oneBlockSize, y: 1 * oneBlockSize },
  { x: 1 * oneBlockSize, y: (map.length - 2) * oneBlockSize },
  { x: (map[0].length - 2) * oneBlockSize, y: oneBlockSize },
  {
    x: (map[0].length - 2) * oneBlockSize,
    y: (map.length - 2) * oneBlockSize,
  }
];

```

```

    },
  ];

  // for (let i = 0; i < map.length; i++) {
  //   for (let j = 0; j < map[0].length; j++) {
  //     map[i][j] = 2;
  //   }
  // }

  let createNewPacman = () => {
    pacman = new Pacman(
      oneBlockSize,
      oneBlockSize,
      oneBlockSize,
      oneBlockSize,
      oneBlockSize / 5
    );
  };

  let gameLoop = () => {
    update();
    draw();
  };

  let gameInterval = setInterval(gameLoop, 1000 / fps);

```

```

let restartPacmanAndGhosts = () => {

  createNewPacman();

  createGhosts();

};

```

```

let onGhostCollision = () => {

  lives--;

  restartPacmanAndGhosts();

  if (lives == 0) {

  }

};

```

```

let update = () => {

  pacman.moveProcess();

  pacman.eat();

  updateGhosts();

  if (pacman.checkGhostCollision(ghosts)) {

    onGhostCollision();

  }

};

```

```

let drawFoods = () => {

  for (let i = 0; i < map.length; i++) {

    for (let j = 0; j < map[0].length; j++) {

      if (map[i][j] == 2) {

```

```

    createRect(
      j * oneBlockSize + oneBlockSize / 3,
      i * oneBlockSize + oneBlockSize / 3,
      oneBlockSize / 3,
      oneBlockSize / 3,
      "#FEB897"
    );
  }
}
};

```

```

let drawRemainingLives = () => {
  canvasContext.font = "20px Emulogic";
  canvasContext.fillStyle = "white";
  canvasContext.fillText("Lives: ", 220, oneBlockSize * (map.length + 1));

  for (let i = 0; i < lives; i++) {
    canvasContext.drawImage(
      pacmanFrames,
      2 * oneBlockSize,
      0,
      oneBlockSize,
      oneBlockSize,
      350 + i * oneBlockSize,

```

```

    oneBlockSize * map.length + 2,
    oneBlockSize,
    oneBlockSize
  );
}
};

```

```

let drawScore = () => {
  canvasContext.font = "20px Emulogic";
  canvasContext.fillStyle = "white";
  canvasContext.fillText(
    "Score: " + score,
    0,
    oneBlockSize * (map.length + 1)
  );
};

```

```

let draw = () => {
  canvasContext.clearRect(0, 0, canvas.width, canvas.height);
  createRect(0, 0, canvas.width, canvas.height, "black");
  drawWalls();
  drawFoods();
  drawGhosts();
  pacman.draw();
  drawScore();
}

```

```

drawRemainingLives();

};

let drawWalls = () => {
  for (let i = 0; i < map.length; i++) {
    for (let j = 0; j < map[0].length; j++) {
      if (map[i][j] == 1) {
        createRect(
          j * oneBlockSize,
          i * oneBlockSize,
          oneBlockSize,
          oneBlockSize,
          "#342DCA"
        );
        if (j > 0 && map[i][j - 1] == 1) {
          createRect(
            j * oneBlockSize,
            i * oneBlockSize + wallOffset,
            wallSpaceWidth + wallOffset,
            wallSpaceWidth,
            wallInnerColor
          );
        }

        if (j < map[0].length - 1 && map[i][j + 1] == 1) {

```

```

createRect(
    j * oneBlockSize + wallOffset,
    i * oneBlockSize + wallOffset,
    wallSpaceWidth + wallOffset,
    wallSpaceWidth,
    wallInnerColor
);
}

```

```

if (i < map.length - 1 && map[i + 1][j] == 1) {
    createRect(
        j * oneBlockSize + wallOffset,
        i * oneBlockSize + wallOffset,
        wallSpaceWidth,
        wallSpaceWidth + wallOffset,
        wallInnerColor
    );
}

```

```

if (i > 0 && map[i - 1][j] == 1) {
    createRect(
        j * oneBlockSize + wallOffset,
        i * oneBlockSize,
        wallSpaceWidth,
        wallSpaceWidth + wallOffset,

```

```

        wallInnerColor
    );
}
}
}
}
};

```

```

let createGhosts = () => {
    ghosts = [];
    for (let i = 0; i < ghostCount * 2; i++) {
        let newGhost = new Ghost(
            9 * oneBlockSize + (i % 2 == 0 ? 0 : 1) * oneBlockSize,
            10 * oneBlockSize + (i % 2 == 0 ? 0 : 1) * oneBlockSize,
            oneBlockSize,
            oneBlockSize,
            pacman.speed / 2,
            ghostImageLocations[i % 4].x,
            ghostImageLocations[i % 4].y,
            124,
            116,
            6 + i
        );
        ghosts.push(newGhost);
    }
}

```

```

};

createNewPacman();

createGhosts();

gameLoop();

window.addEventListener("keydown", (event) => {

  let k = event.keyCode;

  setTimeout(() => {

    if (k === 37 || k === 65) {

      // left arrow or a

      pacman.nextDirection = DIRECTION_LEFT;

    } else if (k === 38 || k === 87) {

      // up arrow or w

      pacman.nextDirection = DIRECTION_UP;

    } else if (k === 39 || k === 68) {

      // right arrow or d

      pacman.nextDirection = DIRECTION_RIGHT;

    } else if (k === 40 || k === 83) {

      // bottom arrow or s

      pacman.nextDirection = DIRECTION_BOTTOM;

    }

  }, 1);

});

```

7.3 GHOST.JS

```
class Ghost {
  constructor(
    x,
    y,
    width,
    height,
    speed,
    imageX,
    imageY,
    imageWidth,
    imageHeight,
    range
  ) {
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
    this.speed = speed;
    this.direction = DIRECTION_RIGHT;
    this.imageX = imageX;
    this.imageY = imageY;
    this.imageHeight = imageHeight;
    this.imageWidth = imageWidth;
    this.range = range;
    this.randomTargetIndex = parseInt(Math.random() * 4);
    this.target = randomTargetsForGhosts[this.randomTargetIndex];
    setInterval(() => {
      this.changeRandomDirection();
    }, 10000);
  }

  isInRange() {
    let xDistance = Math.abs(pacman.getMapX() - this.getMapX());
    let yDistance = Math.abs(pacman.getMapY() - this.getMapY());
    if (
      Math.sqrt(xDistance * xDistance + yDistance * yDistance) <=
        this.range
    ) {
      return true;
    }
  }
}
```

```

    return false;
}

changeRandomDirection() {
    let addition = 1;
    this.randomTargetIndex += addition;
    this.randomTargetIndex = this.randomTargetIndex % 4;
}

moveProcess() {
    if (this.isInRange()) {
        this.target = pacman;
    } else {
        this.target = randomTargetsForGhosts[this.randomTargetIndex];
    }
    this.changeDirectionIfPossible();
    this.moveForwards();
    if (this.checkCollisions()) {
        this.moveBackwards();
        return;
    }
}

moveBackwards() {
    switch (this.direction) {
        case 4: // Right
            this.x -= this.speed;
            break;
        case 3: // Up
            this.y += this.speed;
            break;
        case 2: // Left
            this.x += this.speed;
            break;
        case 1: // Bottom
            this.y -= this.speed;
            break;
    }
}

moveForwards() {
    switch (this.direction) {
        case 4: // Right

```

```

        this.x += this.speed;
        break;
    case 3: // Up
        this.y -= this.speed;
        break;
    case 2: // Left
        this.x -= this.speed;
        break;
    case 1: // Bottom
        this.y += this.speed;
        break;
    }
}

checkCollisions() {
    let isCollided = false;
    if (
        map[parseInt(this.y / oneBlockSize)][
            parseInt(this.x / oneBlockSize)
        ] === 1 ||
        map[parseInt(this.y / oneBlockSize + 0.9999)][
            parseInt(this.x / oneBlockSize)
        ] === 1 ||
        map[parseInt(this.y / oneBlockSize)][
            parseInt(this.x / oneBlockSize + 0.9999)
        ] === 1 ||
        map[parseInt(this.y / oneBlockSize + 0.9999)][
            parseInt(this.x / oneBlockSize + 0.9999)
        ] === 1
    ) {
        isCollided = true;
    }
    return isCollided;
}

changeDirectionIfPossible() {
    let tempDirection = this.direction;
    this.direction = this.calculateNewDirection(
        map,
        parseInt(this.target.x / oneBlockSize),
        parseInt(this.target.y / oneBlockSize)
    );
    if (typeof this.direction === "undefined") {

```

```

    this.direction = tempDirection;
    return;
}
if(
    this.getMapY() != this.getMapYRightSide() &&
    (this.direction == DIRECTION_LEFT ||
    this.direction == DIRECTION_RIGHT)
) {
    this.direction = DIRECTION_UP;
}
if(
    this.getMapX() != this.getMapXRightSide() &&
    this.direction == DIRECTION_UP
) {
    this.direction = DIRECTION_LEFT;
}
this.moveForwards();
if (this.checkCollisions()) {
    this.moveBackwards();
    this.direction = tempDirection;
} else {
    this.moveBackwards();
}
console.log(this.direction);
}

```

```

calculateNewDirection(map, destX, destY) {
    let mp = [];
    for (let i = 0; i < map.length; i++) {
        mp[i] = map[i].slice();
    }
}

```

```

let queue = [
    {
        x: this.getMapX(),
        y: this.getMapY(),
        rightX: this.getMapXRightSide(),
        rightY: this.getMapYRightSide(),
        moves: [],
    },
];
while (queue.length > 0) {
    let popped = queue.shift();
}

```

```

    if (poped.x === destX && poped.y === destY) {
        return poped.moves[0];
    } else {
        mp[poped.y][poped.x] = 1;
        let neighborList = this.addNeighbors(poped, mp);
        for (let i = 0; i < neighborList.length; i++) {
            queue.push(neighborList[i]);
        }
    }
}

return 1; // direction
}

addNeighbors(poped, mp) {
    let queue = [];
    let numOfRows = mp.length;
    let numOfColumns = mp[0].length;

    if (
        poped.x - 1 >= 0 &&
        poped.x - 1 < numOfRows &&
        mp[poped.y][poped.x - 1] !== 1
    ) {
        let tempMoves = poped.moves.slice();
        tempMoves.push(DIRECTION_LEFT);
        queue.push({ x: poped.x - 1, y: poped.y, moves: tempMoves });
    }

    if (
        poped.x + 1 >= 0 &&
        poped.x + 1 < numOfRows &&
        mp[poped.y][poped.x + 1] !== 1
    ) {
        let tempMoves = poped.moves.slice();
        tempMoves.push(DIRECTION_RIGHT);
        queue.push({ x: poped.x + 1, y: poped.y, moves: tempMoves });
    }

    if (
        poped.y - 1 >= 0 &&
        poped.y - 1 < numOfColumns &&
        mp[poped.y - 1][poped.x] !== 1
    ) {
        let tempMoves = poped.moves.slice();

```

```

    tempMoves.push(DIRECTION_UP);
    queue.push({ x: popped.x, y: popped.y - 1, moves: tempMoves });
  }
  if (
    popped.y + 1 >= 0 &&
    popped.y + 1 < numOfColumns &&
    mp[popped.y + 1][popped.x] != 1
  ) {
    let tempMoves = popped.moves.slice();
    tempMoves.push(DIRECTION_BOTTOM);
    queue.push({ x: popped.x, y: popped.y + 1, moves: tempMoves });
  }
  return queue;
}

getMapX() {
  let mapX = parseInt(this.x / oneBlockSize);
  return mapX;
}

getMapY() {
  let mapY = parseInt(this.y / oneBlockSize);
  return mapY;
}

getMapXRightSide() {
  let mapX = parseInt((this.x * 0.99 + oneBlockSize) / oneBlockSize);
  return mapX;
}

getMapYRightSide() {
  let mapY = parseInt((this.y * 0.99 + oneBlockSize) / oneBlockSize);
  return mapY;
}

changeAnimation() {
  this.currentFrame =
    this.currentFrame == this.frameCount ? 1 : this.currentFrame + 1;
}

draw() {
  canvasContext.save();
  canvasContext.drawImage(

```

```

    ghostFrames,
    this.imageX,
    this.imageY,
    this.imageWidth,
    this.imageHeight,
    this.x,
    this.y,
    this.width,
    this.height
);
// canvasContext.restore();
// canvasContext.beginPath();
// canvasContext.strokeStyle = "red";
// canvasContext.arc(
//   this.x + oneBlockSize / 2,
//   this.y + oneBlockSize / 2,
//   this.range * oneBlockSize,
//   0,
//   2 * Math.PI
// );
canvasContext.stroke();
}
}

let updateGhosts = () => {
  for (let i = 0; i < ghosts.length; i++) {
    ghosts[i].moveProcess();
  }
};

let drawGhosts = () => {
  for (let i = 0; i < ghosts.length; i++) {
    ghosts[i].draw();
  }
};

```